

实验四：数值微分、数值积分和微分方程数值解

实验性质

本实验为验证性和设计性实验。

实验目标

1. 掌握用于数值微分和数值积分的各类算法
2. 掌握用于求解微分方程的常用数值解法
3. 生成实验用的数据和函数
4. 学会使用上述方法求解实际问题

先决条件

具有 Python 编程的基本知识，了解与本课程有关的概念

实验材料

- 一台带有 Python 解释器的计算机
- 可以运行 Jupyter Lab 的实验环境

实验过程说明

数值微分和数值积分实验

生成实验数据

下面的示例代码可以从你的学号生成用来练习求根的函数及其导函数。

```
1  def generate_func(school_number):
2      '''
3      根据学生学号生成抛物线函数及其导函数、迭代函数。
4
5      参数：
6      school_number (int): CIT学号，应为8位数字。
7
8      返回：
9      tuple: 包含三个函数
10         第一个是生成的抛物线函数（目标函数）
11         第二个是该函数的导函数
12         第三个是从函数推导的不动点迭代函数
13
14     示例：
15     my_f, my_df, my_g = generate_func(22030501)
16     然后可以使用 my_f(x) 和 my_df(x) 来计算函数值和导函数值。
17     在不动点迭代时，可以用 my_g(x) 来进行不动点迭代。
```

```

18     '''
19
20     # 确保输入是一个8位的整数
21     school_number_str = str(school_number)
22     if len(school_number_str) != 8:
23         raise ValueError("学号应为8位数字")
24
25     # 从学号中提取系数
26     a = int(school_number_str[3])
27     b = int(school_number_str[5])
28     c = int(school_number_str[6:]) / 10
29
30     # 定义一个抛物线函数（目标函数）及其导函数
31     f = lambda x: a * x**2 + b * x - c
32     df = lambda x: 2 * a * x + b
33
34     # 定义从目标函数推得的一个用于不动点迭代的函数
35     g = lambda x: c / (a * x + b)
36
37     return f, df, g

```

测试用例

```

1 school_number = 22030501 # 换成你的学号
2 my_func, my_dfunc, my_iter = generate_func(school_number)

```

下面的实验中，我们用 `my_func` 指代需要求导的函数，用 `my_dfunc` 指代需要求积的函数。

数值微分

数值微分是一种计算函数导数的近似方法，特别用于那些无法直接求导或解析求导困难的函数。这些方法基于函数在一系列点上的已知值，通过构造导数的近似表达式来实现。

基本思想是用函数在某点附近的有限差分来近似导数。根据所使用的点的数量和它们的位置，有不同的差分方案，如向前差分、向后差分和中心差分。

向前差分

向前差分是最简单的差分形式，使用当前点和其后一个点来近似导数，即

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

用 Python 代码标识这个近似计算公式则是

```

1 def ndiff_forward(f, x=0, h=0.1):
2     return (f(x + h) - f(x)) / h

```

其中， h 是差分的步长。显然，这个公式的误差项是 $O(h)$ 。

向后差分

与向前差分类似，向后差分使用当前点和它的前一个点来近似计算导数，即

$$f'(x) \approx \frac{f(x) - f(x - h)}{h}$$

它同样具有 $O(h)$ 的误差项。

中心差分

它使用当前点的前后两个点来提供一个对称的、更平衡的导数估计，即

$$f'(x) \approx \frac{f(x + h) - f(x - h)}{2h}$$

其误差项为 $O(h^2)$ ，相比于向前差分和向后差分，它提高了精度。实际上，这就是三点公式的“中点式”。

```
1 def ndiff_central(f, x=0, h=0.1):  
2     return (f(x + h) - f(x - h)) / (2 * h)
```

三点公式

与对称的中点式相对应，我们有以下非对称的三点公式，即所谓“端点式”

$$f'(x) \approx \frac{-3f(x) + 4f(x + h) - f(x + 2h)}{2h}$$

误差也是 $O(h^2)$ 。

五点公式

需要非常高精度计算导数的场合，五点公式提供 $O(h^4)$ 的精度，例如中点式的五点公式如下

$$f'(x) \approx \frac{f(x - 2h) - 8f(x - h) + 8f(x + h) - f(x + 2h)}{12h}$$

实验要求

用 Python 分别实现

1. 非对称的三点公式，即“端点式”
2. 对称的五点公式，即“中点式”

的计算函数，并且连同向前差分公式（`ndiff_forward`）一起，近似计算 `my_func` 在 $x = 0$ 处的一阶导数，取步长 $h = 0.1$ 。

数值积分

数值积分是一种计算函数在某一区间上积分的近似方法，适用于那些难以或无法通过解析方法直接计算积分的函数。通常涉及将积分区间划分成较小的部分，然后在这些小区间上使用简单的几何形状（如矩形、梯形、抛物线等）来估计面积。

矩形法

矩形法是最简单的求积方法，其基本思想是用矩形去估算函数在某个区间上围成的面积。通常，矩形的“高”由区间某一端点的函数值决定。

例如，用函数 $f(x)$ 的左端点的在区间 $[a, b]$ 上的矩形积分公式如下

$$\int_a^b f(x)dx \approx (b-a) \cdot f(a)$$

其 Python 代码实现

```
1 def nint_rectangle_left(f, a, b):
2     height = f(a)
3     width = b - a
4     return height * width
```

类似地，如果改用右端点 $f(b)$ ，则只需替换上面公式中的 $f(a)$ 即可。

梯形法

这里仅讨论包括区间端点的两点梯形法，复合梯形公式见“复合积分”。如果将区间端点看作梯形的两个底，那么可以得到比矩形公式更精确一些的梯形公式，即

$$\int_a^b f(x)dx \approx \frac{h}{2} \cdot (f(a) + f(b))$$

用 Python 很容易编程实现

```
1 def nint_trapezoidal_simple(f, a, b):
2     h = b - a
3     return (h / 2) * (f(a) + f(b))
```

注意这里的步长 $h = b - a$ 。

Simpson 法

这里仅讨论包括区间端点在内的简单 Simpson 法，复合 Simpson 公式见“复合积分”。Simpson 法使用二次多项式（抛物线），通过三个点来进一步提高积分估计的精度。

$$\int_a^b f(x)dx \approx \frac{h}{3} \cdot (f(a) + 4f(\frac{a+b}{2}) + f(b))$$

这里 $h = \frac{b-a}{2}$ ，相当于在区间的两个端点之间增加一个中点，来提高精度。

复合积分

对于积分区间较大的情况，使用高阶公式会因为等距节点的高阶插值带来振荡。复合积分技术可以在每一段子区间上都用低阶公式，然后对每段子区间得到的结果求和，最终获得整个区间上的积分近似值。

例如可以把区间 $[a, b]$ 等距剖分成 n 段，在每一段子区间上应用梯形公式，最终得到的复合梯形公式，即

$$\int_a^b f(x)dx \approx \frac{h}{2} \cdot (f(a) + 2 \sum_{i=1}^{n-1} f((a+ih)) + f(b))$$

其中 $h = \frac{b-a}{n}$ 。

类似的，如果在每一段子区间上应用 Simpson 公式，我们得到

$$\int_a^b f(x)dx \approx \frac{h}{3} \cdot (f(a) + 2 \sum_{i=1}^{n/2-1} f((a + 2ih)) + 4 \sum_{i=1}^{n/2} f((a + (2i - 1)h)) + f(b))$$

对于复合 Simpson 公式来说，这里的 n 必须是偶数。

我们可以验证，例如，当 $n = 4$ 时，复合 Simpson 公式可以展开为

$$\int_a^b f(x)dx \approx \frac{h}{3} \cdot [(f(a) + 4f(a + h) + f(a + 2h)) + (f(a + 2h) + 4f(a + 3h) + f(b))]$$

它显然是两组简单 Simpson 公式的求和。

实验要求

用 Python 分别实现

1. 简单 Simpson 公式
2. 复合梯形公式

的计算函数，并且连同简单梯形公式（`nint_trapezoidal_simple()`）一起，近似计算 `my_dfunc` 在区间 $[0, 1]$ 上的定积分。对于复合积分公式，取步长 $h = 0.1$ 。

微分方程组

微分方程求解的一般步骤涉及将连续的动力学问题转换为一系列离散步骤，使之可以通过计算机程序实现。该过程主要包括以下几个关键步骤：

1. **建立数学模型**：根据实际问题定义相关的微分方程。包括确定微分方程的类型（常微分方程、偏微分方程）、系统的初始条件以及边界条件。
2. **选择数值方法**：根据问题的特性选择合适的数值方法。对于 ODE，常用的方法包括 Euler 法、Runge-Kutta 法等。对于 PDE，则可能使用有限差分法、有限元法等。
3. **离散剖分**：将微分方程通过时间或空间的离散转换为代数方程。这一步是数值求解的核心，涉及步长（时间或空间）的选择，直接影响计算的精度和稳定性。
4. **编程实现**：将离散后的方程使用具体的计算机程序来实现求解。涉及编写用于迭代计算的循环，处理初始和边界条件，设置停机条件等。
5. **验证求解**：运行程序进行数值求解，并通过分析或与已知解比较来验证求解结果的正确性和稳定性。
6. **分析结果**：分析求解结果，如通过图形可视化呈现动态变化过程，或对结果进行进一步的数学或统计分析，以得到对实际问题的深入理解。

利用 SIR 模型计算流行病扩散

SIR 模型是一种常用的流行病学模型，用于描述易感者(Susceptible)、感染者(Infected)和康复者(Recovered)之间的动态转换。不考虑自然出生率的 SIR 模型的微分方程组如下：

- 易感者变化率： $\frac{dS}{dt} = -\beta SI$
- 感染者变化率： $\frac{dI}{dt} = \beta SI - (\rho + \mu)I$
- 康复者变化率： $\frac{dR}{dt} = \rho I$

其中， β 是局部感染率， ρ 是诊断率， μ 是感染者的死亡率。注意到 $S(t)$ 、 $I(t)$ 和 $R(t)$ 三个函数在方程组中相互依赖。

下面是使用显式 Euler 法来求解 SIR 模型的步骤。

首先，我们通过文献寻找某次流行病爆发时的参数，得到如下信息：

- $\beta = 9.906 \times 10^{-8}$
- $\rho = 0.24$
- $\mu = 0.1$

在这次流行病爆发的开始，我们了解到初始条件是

- $S_0 = 6000000$ ，即总人口中的 600 万人都可能感染
- $I_0 = 245$ ，即总人口中的 245 人已经被感染
- $R_0 = 0$ ，即总人口在初始阶段还没有康复者（假设康复者已经免疫）

根据 SIR 假设，建立微分方程组，并将之以 Python 代码表示

```
1 def sir_model(t, y, beta, rho, mu):
2     S, I, R = y
3     dSdt = -beta * S * I
4     dIdt = beta * S * I - (rho + mu) * I
5     dRdt = rho * I
6     return np.array([dSdt, dIdt, dRdt])
```

显式 Euler 法使用以下近似公式计算

$$w_{i+1} = w_i + hf(t_i, w_i)$$

用 Python 表示就是：

```
1 def euler_step(w, t, h, f, *args):
2     dy = f(t, w, *args)
3     w_next = w + h * dy
4     return w_next
```

其中设置了 `args` 是为了灵活性，方便后面传入模型的“参数”。注意，此处仅定义显示 Euler 的迭代式的表示，并不涉及数值求解时的实际迭代过程。实验时请参考这种设计思想。

下面正式进行迭代求解。我们需要实现一个函数 `solve_sir()` 来管理每一次迭代的结果，它可以传入 `method` 作为迭代方法（因此该参数必须为函数类型），以及必要的模型参数或设置条件。

```
1 import numpy as np
2
3 def solve_sir(method, S0, I0, R0, beta, rho, mu, dt, T=None, obs_length=None):
4
5     # 构建从 0 到 T 的剖分
6     if T is not None and obs_length is not None:
7         # 检查参数 T 和 obs_length 是否兼容
8         if not np.isclose(T / dt + 1, obs_length):
```

```

9         raise ValueError("T 与 obs_length 在给定的 dt 下不兼容")
10        t = np.linspace(0, T, obs_length)
11    elif T is not None:
12        t = np.arange(0, T + dt, dt)
13        obs_length = len(t)
14    elif obs_length is not None:
15        T = dt * (obs_length - 1)
16        t = np.linspace(0, T, obs_length)
17    else:
18        raise ValueError("提供 T 或者 obs_length")
19
20    w = np.zeros((obs_length, 3))
21    w[0, :] = S0, I0, R0
22
23    # 迭代部分
24    for i in range(1, obs_length):
25        w[i] = method(w[i-1], t[i-1], dt, sir_model, beta, rho, mu)
26
27    return t, w

```

注意到我们在函数 `solve_sir()` 中声明了两个数组 `t` 和 `w`，分别表示从初始时刻到终止时刻（即 $[0, T]$ ）的每一个时间步和相应时间步上的近似函数值。如此，函数体内部所要做的无非是用所选 `method` 迭代求解而已。

我们给出参数设置，并将结果用 Matplotlib 画出来。

```

1  beta = 9.906e-8    # 局部感染率
2  rho = 0.24         # 诊断率
3  mu = 0.1           # 感染者的死亡率
4  S0 = 6e6           # 初始易感人群数
5  I0 = 245           # 初始感染人群数
6  R0 = 0             # 初始康复人群数
7  dt = 0.1           # 步长
8
9  t, w = solve_sir(euler_step, S0, I0, R0, beta, rho, mu, dt, T=50)
10
11  import matplotlib.pyplot as plt
12
13  plt.figure(figsize=(10, 6))
14  plt.plot(t, w[:, 0], label='$S(t)$')
15  plt.plot(t, w[:, 1], label='$I(t)$')
16  plt.plot(t, w[:, 2], label='$R(t)$')
17  plt.plot(t, np.sum(w, axis=1), label='Total')
18  plt.title('SIR Model (Euler)')
19  plt.xlabel('Time')
20  plt.ylabel('Population')
21  plt.legend()
22  plt.show()

```

注意到，求解 SIR 时，我们代入了函数名 `euler_step`。另请注意，在画图时，除了画出解 `w` 的三个函数分量，还用 `np.sum()` 计算这三个分量的总和，以便观察总人口的变化趋势。我们把 T 设置为 50，可以看作模拟了 50 天的流行病传播过程。

实验要求

1. 使用标准四阶 Runge-Kutta 法替代上面的显式 Euler 法。你需要做的仅仅是仿照示例的 `euler_step()`，实现一个叫做 `rk4_step()` 的函数。请参考课件。
2. 当面临流行病爆发时，不太可能找到现成可用的 SIR 模型参数，我们需要通过卫健委的观测数据来“拟合” SIR 模型，率定出合适的模型参数 (β 、 ρ 和 μ)。现在，已知 50 天内的 $S(t)$ 、 $I(t)$ 和 $R(t)$ 的观察数据是

```
1 S_obs = np.array([6000000, 5998562, 5993524, 5975937, 5915474, 5717962, 5166337,
2 4110774, 2993076, 2298680, 1969285])
3 I_obs = np.array([ 245, 859, 3014, 10515, 36078, 117077, 320886,
593843, 625901, 416714, 216109])
R_obs = np.array([ 0, 580, 2616, 9735, 34370, 116614, 362132,
914560, 1680894, 2318717, 2692835])
```

由于这个采样非常稀疏（相当于每 5 天统计一次人口），为了提高预测精度，我们将会选用在第 1 小题实现的 `rk4_step()` 函数进行预测，而不用示例的 `euler_step()` 函数。此外，要求使用最小二乘来拟合预测值和观测值，从而得到最合适的一组模型参数。请将率定得到的参数 β 、 ρ 和 μ 打印出来，并代入 SIR 形成“预测模型”。最后，画出你的“预测模型”的函数曲线和观测值的数据点。预测模型中的 $S(t)$ 、 $I(t)$ 和 $R(t)$ 三个函数使用虚线表示，观测点使用圆点表示。

模型参数的参考范围：

- β : [1e-10, 1e-2]
- ρ : [1e-4, 5e-1]
- μ : [1e-5, 5e-1]

本实验建议实现步骤：

1. 实现 `rk4_step()` 的函数，并在示例代码的上下文中测试是否正确。
2. 实现最小二乘要用的误差函数。
3. 设置模型参数的迭代范围，并遍历这些范围，得到最优模型参数。也可以试着使用诸如 `scipy.optimize` 中的最优化库函数。
4. 代入最优模型参数，画图。

撰写实验报告

1. 总结该实验，包括实验目标、实验方法和实验过程。
2. 描述实验实施的具体情况和结果，包括遇到的问题和挑战，已经如何解决问题的思路。
3. 分析实验结果，并根据所使用的数据和方法得出结论。
4. 将实验报告誊写到报告书模版文档。